NESTLY · ENGINEERING DOCUMENTATION

High-Level Design

System architecture · AWS deployment · data · consistency · resilience

Contents

1 Architecture goals and principles
2 Requirements and capacity
3 Architecture overview
4 AWS deployment and network
5 Application architecture: modular monolith
6 Data architecture and database scaling
7 Caching layer
8 Booking consistency and concurrency
9 Payments, idempotency and Saga
10 Asynchronous processing, search and media
11 Scaling and traffic management
12 Security and observability
13 Failure handling and disaster recovery
14 CI/CD, evolution and trade-offs
15 Complete system diagram

1. Architecture goals and principles

Nestly prioritizes **correctness on the booking and payment path**, low latency on read-heavy discovery, durable asynchronous processing and an explicit evolution path from a modular monolith to independent services.

Principle	What it means in Nestly
One source of truth	PostgreSQL owns all authoritative state. Redis, OpenSearch and CDN caches are derived and can be rebuilt.
Correctness before speed	Booking and payment use strong consistency and database-enforced invariants. Discovery tolerates staleness.
Stateless compute	API nodes hold no session or durable state, so any node can serve any request and nodes are replaceable.
Fail safe, not fail open	When a dependency fails, the system degrades non-critical features and never risks double-booking or double-charging.
Asynchronous by default for side effects	Notifications, indexing, analytics and provider actions run through SQS with idempotent consumers.
Distribute only on evidence	Start as a modular monolith; extract a service only for measured load, ownership or reliability reasons.

Concern	Technology
Source of truth	PostgreSQL on RDS (Multi-AZ)
Cache / coordination	Redis on ElastiCache
Async messaging	SQS with dead-letter queues
Search	OpenSearch
Media	S3 with CloudFront
Compute	Node/Express containers on EC2 first; ECS/Fargate later

2. Requirements and capacity

Assumption	Value	Design implication
Registered users	1M	Index user lookups; paginate everything
Monthly / daily active users	100K / 10K	Read-heavy; horizontal API scaling is enough
Properties / services	5K / 20K	Small catalog: cacheable, easily indexed
Daily searches	10K	Search cache and index, not database scans
Daily bookings	1K	Low volume, but bursty and contention-prone

What the numbers imply. 10K searches/day averages about 0.12 requests/s and 1K bookings/day about 0.012 requests/s. Even with an assumed 100× burst factor that is roughly 12 and 1.2 requests/s. Raw throughput is not the challenge. The real design problems are **contention** (many users wanting the same date), **correctness** across multiple resources, and **burst behavior**. The architecture is sized for correctness first; scale headroom comes from statelessness and caching. These are design assumptions to be verified with load tests, not production claims.

3. Architecture overview

A request travels through six layers. The complete picture is in the diagram at the end of this document; this table is the reference for what each layer owns.

Layer	Components	Responsibility
Edge	Route 53, CloudFront, API Gateway	DNS, TLS, static and media delivery, throttling, coarse routing
Ingress	ALB, NAT gateway	Health-checked load balancing to private nodes; controlled egress to the payment provider
Compute	Node/Express API (N instances), workers	Business logic in a modular monolith; background jobs and schedulers
Data	RDS PostgreSQL primary and replicas	Authoritative state, transactions, constraints
Coordination & async	ElastiCache Redis, SQS and DLQs	Cache, locks, rate limits; durable decoupled work
Derived stores	OpenSearch, S3	Search index (eventual); object storage for media
Cross-cutting	Secrets Manager, IAM, CloudWatch, OpenTelemetry, GitHub Actions/ECR	Credentials, access, telemetry, delivery

Request paths

Path	Flow	Consistency
Read (browse)	CloudFront → API Gateway → ALB → API → Redis hit, or PostgreSQL then populate Redis	Eventual (cache TTL)
Search	API → OpenSearch (fed from PostgreSQL via outbox and SQS)	Eventual; checkout revalidates
Booking	API → Redis lock (optional) → PostgreSQL transaction → outbox event	Strong
Payment	API → provider (idempotent) ; provider webhook → API → PostgreSQL	Strong, reconciled
Media upload	Browser → presigned URL → S3 directly; read via CloudFront	Eventual; DB stores metadata
Side effects	Outbox → SQS → workers (notify, index, provider actions, analytics)	Eventual, at-least-once

4. AWS deployment and network

Tier	Contents	Exposure and controls
Public edge	Route 53, CloudFront, API Gateway	Internet-facing; TLS; throttling; WAF as an option
Public subnets	ALB, NAT gateway	ALB accepts only from API Gateway / CloudFront; NAT for outbound only
Private app subnets	API and worker instances	No public IPs; security group accepts only from the ALB
Private data subnets	RDS, ElastiCache, OpenSearch	Security groups accept only from app and worker groups
Regional services	S3, SQS, Secrets Manager, CloudWatch	Reached via IAM roles; VPC endpoints to avoid NAT cost where useful

- Deploy across at least two Availability Zones: ALB, app instances and RDS Multi-AZ.
- No static AWS keys: instances use IAM roles; database and provider secrets come from Secrets Manager.
- **API Gateway plus ALB:** both are kept for edge throttling and API controls plus health-checked balancing to private instances. At the stated scale CloudFront directly to the ALB would also work; API Gateway is retained for usage plans and request-level controls and can be dropped later if it adds cost without value.

- Nginx is used only to serve or reverse-proxy the containerized client. It is not added to every request path without a concrete purpose.

5. Application architecture: modular monolith

The API is one deployable Node/Express application organized as domain modules. This gives clear boundaries and fast local calls without the operational cost of many services.

Domain	Modules	Notes
Identity	Auth, Users	Login, tokens, roles
Catalog	Properties, Providers, Services, Events, Packages	Read-heavy, cacheable, indexed for search
Booking core	Availability, Reservations, Orders	Strongly consistent; highest correctness demands
Money	Payments	State machine, idempotency, webhooks, reconciliation
Engagement & ops	Search API, Notifications, Reviews, Admin	Mostly async or read paths

Module rules

- A module owns its tables; other modules use its service interface or events, never its tables directly.
- Cross-module side effects go through the outbox and SQS, not direct calls, so extraction stays cheap.
- Every mutation authorizes and validates before domain logic; critical invariants are re-checked inside the transaction.

Extraction candidates (only when measured need appears)

Candidate	Extract when
Search	Query load or index operations interfere with transactional traffic
Booking / Reservations	Contention or deployment cadence needs isolated scaling
Payments	Compliance, ownership or risk boundary demands separation
Notifications	Delivery volume or provider integrations grow independently
Provider operations	Provider-facing workflows need their own release cycle

6. Data architecture and database scaling

Domain	Entities	Store	Consistency
Identity	users	PostgreSQL	Strong
Inventory	properties, services, availability	PostgreSQL	Strong
Orders	orders, order_items	PostgreSQL	Strong
Reservations	reservations	PostgreSQL	Strong, conflict-safe (constraint)
Payments	payments, payment_transactions, idempotency_keys	PostgreSQL	Strong, idempotent
Experience	events, packages	PostgreSQL + cache + index	Read-heavy, eventual reads
Operations	notifications, audit_logs, payouts	PostgreSQL	Durable; delivery async
Media	image objects	S3 (metadata in PostgreSQL)	Eventual

Scaling path, in order

1. **Indexes and query discipline.** Composite indexes on resource and time range; avoid unbounded scans; pagination everywhere.
2. **Connection management.** Bounded pools per instance. Budget: instances × pool size must stay under the database limit; add RDS Proxy or PgBouncer before connection count becomes the constraint.
3. **Read replicas** for stale-tolerant reads (browse, reviews, analytics). Never for booking-critical decisions because replica lag can hide a fresh reservation.
4. **Partitioning** of large append-heavy tables (audit_logs, notifications) by time.
5. **Sharding** only if a single primary is measurably exhausted; it is deliberately last because it complicates transactions.

7. Caching layer

Pattern: **cache-aside**. The API checks Redis; on a miss it reads PostgreSQL, populates Redis with a TTL and returns.

Cached item	Key	TTL (starting point)	Invalidation
Property detail	`cache:property:{id}`	Minutes	Delete on property update
Search results	`cache:search:{hash}`	Seconds to a minute	TTL only
Service listings	`cache:service:{id}`	Minutes	Delete on service update

- **Never cache** availability used for a booking decision. Availability shown to users is indicative; the transaction decides.
- **Stampede protection:** per-key short lock or request coalescing, plus jittered TTLs.
- **Redis outage:** reads fall through to PostgreSQL (slower, still correct). Rate limiting fails open per a conservative default.
- TTLs are starting points to tune from the measured cache hit ratio.

8. Booking consistency and concurrency

Correctness is enforced in **three independent layers**, so a failure in one cannot cause a double booking.

Layer	Mechanism	Purpose	Is it authoritative?
1 Coordination	Redis lock SET lock:{resource}:{slot} token NX EX ttl	Serialize contenders and cut wasted database work	No: an optimization
2 Transaction	PostgreSQL transaction re-checks and writes the reservation	Atomic check-and-reserve	Yes
3 Constraint	Exclusion constraint on (resource, time range) for active reservations	Database refuses overlapping rows even if 1 and 2 fail	Yes: final guard

Sequence

1. Validate order ownership and input.
2. Try to acquire the Redis lock with a unique token and short TTL. If rejected, return a conflict quickly.
3. Begin a transaction: expire stale holds for this resource, re-check state, insert the reservation as HELD with hold_expires_at.
4. Commit. The constraint guarantees no overlapping active reservation, so a second commit fails cleanly.
5. Release the lock with compare-and-delete (only the owner's token can delete).
6. Continue to payment; on success confirm the reservation, on failure or timeout release it.

> **Design decision.** The hold's expiry lives in PostgreSQL (*hold_expires_at*), not only in a Redis key. A Redis key expiring does not change database state, so an expiry sweeper plus lazy expiry inside the booking transaction reconcile expired holds. This also means a Redis outage does not break correctness: booking can fall back to database-only locking (row lock plus constraint), slower under contention but still safe.

Resource types. Exclusive resources (a property on a date range, a single photographer) use the exclusion constraint. Capacity-based services (a caterer serving up to N events) use a row-locked capacity counter checked in the transaction.

9. Payments, idempotency and Saga

Payment state machine

State	Meaning	Allowed next
CREATED	Record exists	PENDING
PENDING	Waiting to proceed	PROCESSING, FAILED
PROCESSING	Provider call in flight	SUCCESS, FAILED (or stays until reconciled if ambiguous)
SUCCESS	Confirmed by provider	REFUNDED
FAILED	Terminal failure	none
REFUNDED	Successful payment returned	none

Idempotency

- Payment requests carry an Idempotency-Key. A scoped unique key (user, operation, key) plus a request hash means the same key and body returns the original result; the same key with a different body is rejected.
- Webhooks are verified for authenticity, identified by provider event ID (unique constraint) and applied once.
- Ambiguous outcomes (timeouts) leave the payment PROCESSING and a reconciliation job queries the provider until resolved.

Saga for multi-resource orders

Property and service reservations are independent commits, so the workflow is an **orchestrated Saga** with a persisted state. Orchestration was chosen over choreography because there are few steps, the order matters, and one place owning the flow makes failures and compensation easy to reason about.

Step	Action	Compensation if a later step fails
1	Hold property	Release property hold
2	Hold each service (catering, decoration, ...)	Release each service hold
3	Process payment (idempotent)	Void or refund
4	Confirm all reservations, mark order CONFIRMED	Retry until success; if impossible, refund and release

Compensations are idempotent and retried until they succeed. The orchestrator state survives restarts, so an interrupted Saga resumes or compensates on recovery. A transactional outbox writes the state change and its event in one PostgreSQL transaction so an event is never lost between commit and publish.

10. Asynchronous processing, search and media

Queue	Producer	Consumer	Notes
`notifications`	Orders, payments	Notification worker	Email/push; retry with backoff
`provider-actions`	Confirmed orders	Provider worker	Fulfilment requests to providers
`analytics`	API events	Analytics worker	Loss-tolerant
`search-index`	Outbox publisher	Indexer	Upsert by document ID
`*-dlq`	SQS redrive	Operators	Alarm on any depth above zero

- SQS is **at-least-once**: every consumer is idempotent (dedupe on eventId or natural key).
- Retries with backoff; after the maximum receive count messages move to the DLQ. Queue depth and DLQ depth are alarmed.
- **Search:** PostgreSQL change → outbox → SQS → indexer → OpenSearch upsert. The index is eventually consistent; checkout always revalidates against PostgreSQL. If OpenSearch is down, search degrades and booking is unaffected.
- **Media:** the API issues a presigned S3 URL; the browser uploads directly; CloudFront serves reads. Large files never pass through the API; PostgreSQL keeps metadata.

11. Scaling and traffic management

Layer	Approach	Signal to scale
API	Add stateless instances behind the ALB; autoscale	CPU, request latency P95, ALB target response time
Workers	Scale by queue depth, independently of the API	Oldest message age, queue depth
PostgreSQL	Bigger instance, then replicas, then partitioning	CPU, connections, replica lag, slow queries
Redis	Larger node or cluster mode	Memory, evictions, CPU
OpenSearch	More or larger data nodes	Query latency, JVM pressure

- Rate limits at API Gateway (coarse) and in the application via Redis counters (per user or route) protect the database from bursts.
- Timeouts, bounded retries with jitter and circuit breakers around the payment provider and other dependencies prevent retry storms.
- Load shedding: return fast 429/503 rather than queueing unboundedly under overload.

12. Security and observability

Layer	Controls
Transport & edge	TLS everywhere, CORS restricted to known origins, Helmet headers, throttling, optional WAF
Identity & access	Hashed passwords, short-lived JWT access tokens, server-side RBAC (Customer, Host, Provider, Admin)
Application	Schema validation on all input, parameterized queries, no stack traces or SQL in errors
Data	Private subnets, security-group allow-lists, encryption at rest and in transit, least-privilege IAM roles
Secrets	Secrets Manager; no secrets in code, images or logs
Audit	Admin and sensitive actions written to audit_logs with actor, entity and time

Signal	Examples	Alarm on
Golden signals	RPS, P50/P95/P99 latency, error rate	Error rate or P95 breaching budget
Booking health	Booking success rate, lock contention, conflicts, hold expiries	Sudden drop in success rate
Payment health	Failure rate, PROCESSING age, reconciliation backlog	Payments stuck beyond threshold
Dependencies	Cache hit ratio, DB connections and CPU, replica lag	Connections near limit; lag above threshold
Async	Queue depth, oldest message age, DLQ depth	Any DLQ message; growing backlog

Every request carries a request ID that appears in structured logs, traces (OpenTelemetry) and error responses, so a failed booking can be traced end to end.

13. Failure handling and disaster recovery

Failure	Behavior	Recovery
Node crash	ALB stops routing to it; no local durable state is lost	Replacement instance from health check / autoscaling
Redis outage	Cache reads fall through to PostgreSQL; booking uses database-only locking	Redis returns; caches warm up
DB primary failure	Multi-AZ failover; brief write errors	Retry idempotent operations; connections re-established
Replica lag	Booking-critical reads stay on the primary	Lag alarm; scale or fix replica
Payment timeout	Payment stays PROCESSING; holds kept until expiry	Reconciliation resolves via provider
Duplicate webhook	Unique provider event ID; state applied once	None needed
Worker crash	Message becomes visible again; retried	After max retries goes to DLQ; inspect and redrive
Search outage	Search degrades; browse and booking continue	Rebuild index from PostgreSQL
S3 or CDN issue	Uploads retried; DB keeps metadata	Re-upload or re-issue URL
Bad deployment	Health checks fail the rollout	Automatic rollback to previous image

Disaster recovery

- RDS automated backups with point-in-time recovery; Multi-AZ for zone failure. Recovery objectives (RPO and RTO) are targets to set with the business and then rehearse with a restore drill.
- Redis and OpenSearch are rebuildable from PostgreSQL; SQS messages are durable within their retention period.
- Infrastructure defined as code so an environment can be recreated; S3 versioning for media.

14. CI/CD, evolution and trade-offs

GitHub Actions runs tests, builds Docker images, pushes to ECR and deploys. Database migrations are versioned and backward-compatible (expand, migrate, contract) so a rollback never meets an incompatible schema. Health checks gate every rollout.

Evolution roadmap

Stage	Architecture	Trigger to move on
1	Modular monolith, PostgreSQL	Baseline
2	+ Redis cache and locks, SQS workers	Read latency or side-effect coupling
3	+ Multiple API instances behind ALB	CPU or availability needs
4	+ Read replicas, OpenSearch	Read load or search quality
5	Extract hot domains (search, booking, payments)	Measured bottleneck or ownership boundary
6	Independent services	Team and reliability boundaries justify the cost

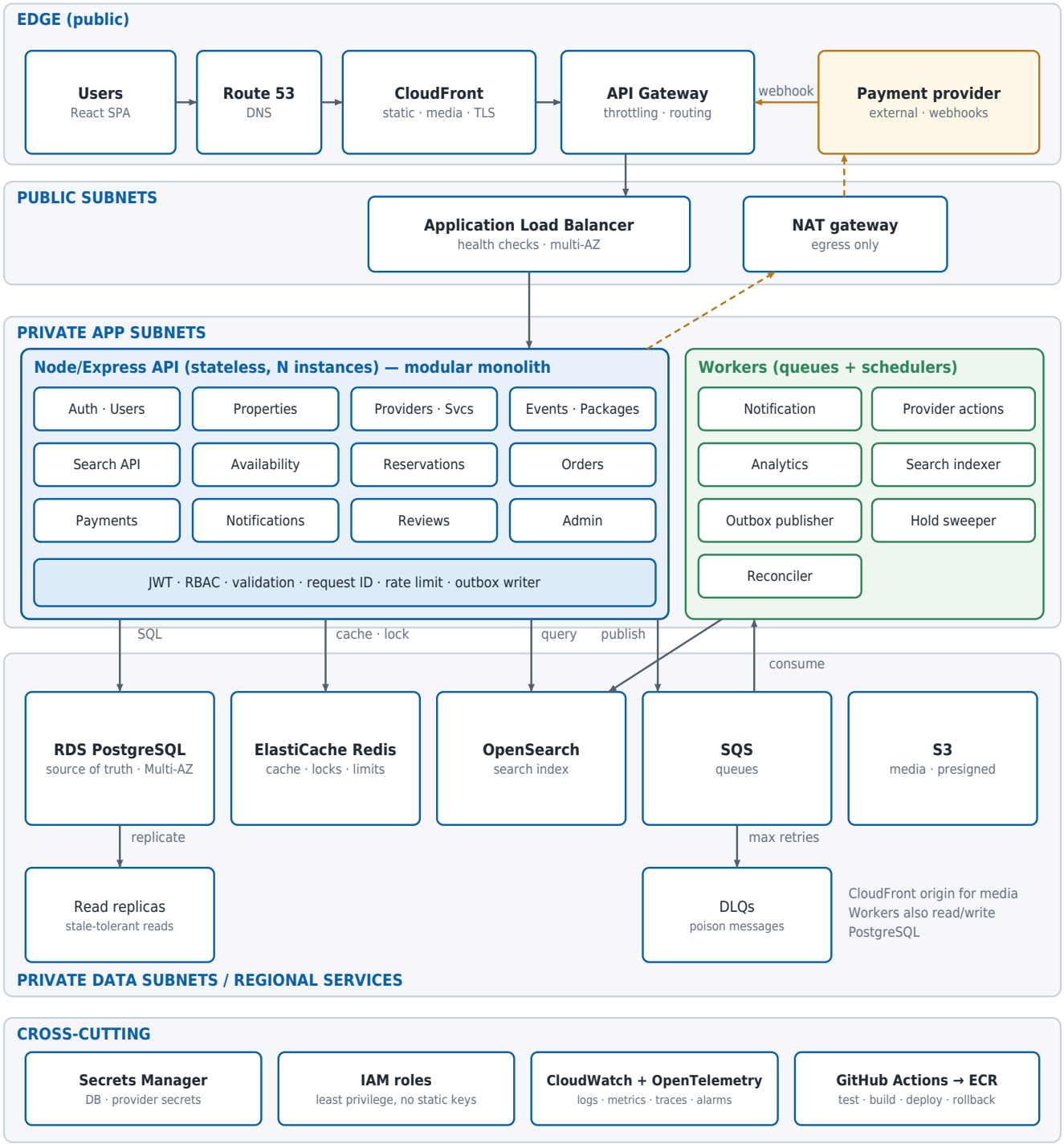
Key trade-offs

Decision	Chosen	Gain	Cost
Database	PostgreSQL	Transactions, constraints, relational model	Vertical scaling limits
Coordination	Redis lock + DB constraint	Fewer conflicts, correctness kept in DB	Two mechanisms to operate
Async	SQS	Durable, decoupled, retried	Eventual completion; idempotency required
Search	OpenSearch	Rich search	Eventual consistency; extra system
Media	S3 + CloudFront	Offloads API; cheap and fast	Object workflow, presigned URLs
Compute	EC2 first, ECS later	Simple to start	Orchestration deferred
Architecture	Modular monolith	Low distributed complexity	Shared deploy and scaling unit
Scaling data	Replicas before sharding	Cheap read scaling	Replica lag; sharding later is harder

Interview emphasis: Redis vs database-only locking, strong vs eventual consistency, SQS vs synchronous calls, replicas vs sharding, monolith vs microservices, and safe behavior when each dependency fails.

15. Complete system diagram

One view of every component, its network placement and the main flows. Read top to bottom: edge, ingress, compute, data.



HOW TO READ THIS

PostgreSQL is the only source of truth. Redis, OpenSearch, S3 and CDN are derived or coordination stores.

Booking path: API → Redis lock (optional) → PostgreSQL transaction + exclusion constraint → outbox → SQS → workers.

Payment path: API → provider (idempotency key); provider webhook → API Gateway → API → PostgreSQL.

Solid arrow = internal call · dashed amber = external/egress · search, cache and notifications are eventually consistent.

Failure rule: cache/search/queue outages degrade features; they never allow a double booking or double charge.